

```

1  /* =====
2      uebersichtsrouten.js – Übersichtsakt und Kapitel-Navigation
3
4      Zeichnet im Übersichtsakt die Routen 02–18 nacheinander auf die rausgezoomte
5      Karte, bei geöffnetem Kapitel stattdessen dessen genaue Route, Kreise und
6      Annotation. Hält zoomedKapitel / kapitelZoomAmount / kapitelHover – nur hier
7      geschrieben, nach aussen Lesebindung. Rest: docs/architektur.md.
8  ===== */
9
10 // --- Modulkapselung -----
11 // 6 von 16 Namen intern, 10 exportiert. Konvention: docs/architektur.md.
12 (function () {
13
14     let zoomedKapitel = null;      // z.B. '03', oder null (Übersicht)
15     let kapitelZoomAmount = 0;    // 0 = Übersicht, 1 = voll in Kapitelausschnitt gezoomt
16     let kapitelHover = null;      // Kapitelnummer unter der Maus (fürs Cursor/Highlight)
17
18     // Teilt den Übersichtsakt in eine Scheibe je Kapitel, nach Routenlänge
19     // gewichtet (sonst 27-fache Tempo-Schwankung) plus festem Grundanteil.
20     const KAPITEL_SCHEIBE_GRUNDANTEIL = 0.45; // Anteil des Akts, der gleichmässig verteilt wird
21     let scheibenCache = null;
22
23     function kapitelScheiben() {
24         if (scheibenCache) return scheibenCache;
25         let liste = Object.keys(uebersichtsRouten || {}).sort();
26         if (!liste.length) return [];
27         let laengen = liste.map(nr => uebersichtsRouten[nr].length);
28         let summe = laengen.reduce((a, b) => a + b, 0) || 1;
29         let grund = KAPITEL_SCHEIBE_GRUNDANTEIL / liste.length;
30         let rest = 1 - KAPITEL_SCHEIBE_GRUNDANTEIL;
31         let anteile = liste.map((nr, i) => grund + rest * laengen[i] / summe);
32         // Alle Scheiben leicht stauchen, damit das Abkühlfenster des letzten
33         // Kapitels nicht hinter dem geklemmten Aktende liegt.
34         let stauchung = 1 / (1 + anteile[anteile.length - 1] * KAPITEL_NACHGLUEHEN);
35         let scheiben = [];
36         let kum = 0;
37         liste.forEach((nr, i) => {
38             let anteil = anteile[i] * stauchung;
39             scheiben.push({ nr, von: kum, bis: kum + anteil });
40             kum += anteil;
41         });
42         scheibenCache = scheiben;
43         return scheibenCache;
44     }
45
46     // 1 während der eigenen Scheibe, danach über KAPITEL_NACHGLUEHEN auf 0.
47     // Route und Badge wechseln gemeinsam von Hoverfarbe auf Routengold.
48     const KAPITEL_NACHGLUEHEN = 0.05; // Anteil einer Scheibe für den Übergang
49
50     function kapitelHitze(fortschritt, scheibe) {
51         if (!scheibe) return 0;
52         let breite = scheibe.bis - scheibe.von;
53         return 1 - constrain(map(fortschritt, scheibe.bis, scheibe.bis + breite * KAPITEL_NACHGLUEHEN, 0, 1), 0, 1);
54     }
55
56     function zeichneUebersichtsrouten(bbox, alpha, fortschritt) {
57         // Ausserhalb des Akts gibt es keine Hover-Ziele. Rücksetzer steht hier,
58         // weil nur hier die Geometrie der Startpunkte bekannt ist.
59         if (fortschritt <= 0) {
60             kapitelHover = null;
61             cursor(ARROW);
62             return { aktuelleAnnotationZoom: null };
63         }
64
65         noFill();

```

```

66     strokeWeight(2);
67
68     let kapitelListe = Object.entries(uebersichtsrouten).sort((a, b) => parseInt(a[0]) -
    parseInt(b[0]));
69     let n = kapitelListe.length;
70
71     // Geöffnetes Kapitel bekommt den vollen Akt als Reveal-Skala, nicht nur
72     // seine Scheibe. Anfang gehört dem Einstiegstext.
73     let zoomedLokalerFortschritt = constrain(
74         map(fortschritt, KAPITEL_EINSTIEG_SCROLL_ENDE, 1, 0, 1), 0, 1);
75     let aktuelleAnnotationZoom = null; // Rückgabewert für die Annotationsbox in draw()
76
77     // Im Kapitel-Zoom blenden alle anderen Übersichtsrouten mit
78     // kapitelZoomAmount aus.
79     kapitelListe.forEach(([kapitelNr, punkte], i) => {
80         let scheibe = kapitelScheiben()[i];
81         let lokalerFortschritt = scheibe
82             ? constrain(map(fortschritt, scheibe.von, scheibe.bis, 0, 1), 0, 1) : 0;
83         if (lokalerFortschritt <= 0) return;
84
85         // Übersichtslinie und Kapitelroute sind dieselben Punkte. Beim eigenen
86         // Zoom hier überspringen, unten mit mapOffsetX gezeichnet – sonst doppelt.
87         if (kapitelNr === zoomedKapitel && kapitelZoomAmount > 0.001) return;
88
89         let routenAlpha = (zoomedKapitel && kapitelNr !== zoomedKapitel)
90             ? alpha * (1 - kapitelZoomAmount)
91             : alpha;
92         if (routenAlpha <= 0) return;
93         // Wachsende Route in Hoverfarbe, danach auf Gold abkühlend.
94         let hitze = zoomedKapitel ? 0 : kapitelHitze(fortschritt, scheibe);
95         stroke(lerp(ROUTE_COLOR_RGB.r, FWERT_COLOR_RGB.r, hitze),
96             lerp(ROUTE_COLOR_RGB.g, FWERT_COLOR_RGB.g, hitze),
97             lerp(ROUTE_COLOR_RGB.b, FWERT_COLOR_RGB.b, hitze), routenAlpha);
98         let anzahl = Math.max(1, Math.round(lokalerFortschritt * punkte.length));
99         beginShape();
100        for (let j = 0; j < anzahl; j++) {
101            let p = lonLatToScreen(punkte[j][0], punkte[j][1], bbox, 0, 0); // zentrierte Übersichtskarte,
    kein mapOffsetX
102            vertex(p.x, p.y);
103        }
104        endShape();
105    });
106
107    // Genaue Route des gezoomten Kapitels, mit fixem mapOffsetX/Y statt dem
108    // chl-Blend. Erst nach dem Einstiegstext (KAPITEL_EINSTIEG_SCROLL_ENDE).
109    let kapitelEinstiegAbgeschlossen = fortschritt >= KAPITEL_EINSTIEG_SCROLL_ENDE;
110    if (zoomedKapitel && kapitelZoomAmount > 0.001 && kapitelEinstiegAbgeschlossen) {
111        let daten = datenFuerKapitel(zoomedKapitel);
112        // routenPfadDetail statt routenPunkte: Letzteres ist auf 1 Punkt je
113        // Annotation komprimiert, die Linie sähe wie eine Luftlinie aus.
114        let routenLinie = (daten && daten.routenPfadDetail && daten.routenPfadDetail.length > 1)
115            ? daten.routenPfadDetail
116            : (daten && daten.routenPunkte);
117        if (routenLinie && routenLinie.length > 1) {
118            // routenPfadKumulativ bindet das Wachstum an die Annotationsfolge
119            // (Stop-and-go statt proportional zum Scrollfortschritt).
120            let kumulativ = daten && daten.routenPfadKumulativ;
121            let upToIndex;
122            if (kumulativ && kumulativ.length === (daten.annotations || []).length) {
123                // Zwischen den kumulativen Pfad-Indizes interpolieren, damit der
124                // Ortswechsel-Sprung weich wächst statt aufzupoppen.
125                let stelle = constrain(zoomedLokalerFortschritt * (kumulativ.length - 1), 0,
    kumulativ.length - 1);
126                let i0 = Math.floor(stelle), i1 = Math.min(kumulativ.length - 1, i0 + 1);
127                let frac = stelle - i0;
128                upToIndex = Math.round(lerp(kumulativ[i0], kumulativ[i1], frac));

```

```

129     } else {
130         upToIndex = Math.round(zoomedLokalerFortschritt * (routenLinie.length - 1));
131     }
132     if (upToIndex >= 1) {
133         // Strichstärke 10 wie Kapitel 1s Route, hier ohne Rauszoom-Phase.
134         zeichneRoute(routenLinie, upToIndex, bbox, 3, mapOffsetX, mapOffsetY, kapitelZoomAmount);
135     }
136 }
137
138 // Wachsende Kreise und aktuelle Annotation wie bei Kapitel 1, aber aus
139 // zoomedLokalerFortschritt statt aus eigenen Scroll-Meilensteinen.
140 if (daten && daten.annotationen && daten.annotations.length) {
141     let annIndexZoom = Math.min(daten.annotations.length - 1, Math.floor(zoomedLokalerFortschritt
* daten.annotations.length));
142     let aktuelleAnnZoom = daten.annotations[annIndexZoom];
143     // ACHTUNG annIndexZoom, nicht revealIndex: bei Kapitel 3 ist
144     // revealIndex nicht die Array-Position, ortRuns[].revealIndex aber
145     // schon – die Kreise erscheinen sonst zu spät oder nie.
146     let punktIndexZoom = aktuelleAnnZoom.vorRoutenstart ? 0 : annIndexZoom;
147     zeichneKreiseOrtRuns(punktIndexZoom, annIndexZoom, bbox, mapOffsetX, mapOffsetY, daten);
148     aktuelleAnnotationZoom = aktuelleAnnZoom;
149 }
150 }
151
152 // Startpunkt und Kapitelnummer je Route. Kapitel mit eigener Ansicht sind
153 // klickbar (kapitelHatEigeneAnsicht).
154 noStroke();
155 textFont(SCHRIFT_SANS); // wie .annotation-tag (var(--sans)) und die Kreis-Labels
156 textStyle(BOLD); // .annotation-tag ist font-weight: 700
157 textAlign(LEFT, CENTER);
158 textSize(11);
159 kapitelHover = null;
160
161 // Mehrere Kapitel teilen sich Startpunkte (02/10, 07/11 – echte Orte,
162 // keine Datenfehler); ohne Versatz überdeckt das höhere Badge das tiefere.
163 // ACHTUNG nach ABSTAND gruppieren, nicht nach gleicher Koordinate: Kapitel
164 // 18 liegt 0.24 m neben 08/09, auf dem Bildschirm derselbe Pixel.
165 const DUP_TOLERANZ = 0.0003; // Grad, rund 25 m – kleinster echter Abstand ist 223 m
166 let startDupGruppen = {};
167 let dupSchluessel = {};
168 kapitelListe.forEach(([kapitelNr, punkte]) => {
169     let [lon, lat] = punkte[0];
170     let key = Object.keys(startDupGruppen).find(k => {
171         let [kl, kb] = k.split(',').map(Number);
172         return Math.abs(kl - lon) < DUP_TOLERANZ && Math.abs(kb - lat) < DUP_TOLERANZ;
173     });
174     if (!key) key = lon + ',' + lat;
175     (startDupGruppen[key] = startDupGruppen[key] || []).push(kapitelNr);
176     dupSchluessel[kapitelNr] = key;
177 });
178
179 kapitelListe.forEach(([kapitelNr, punkte], i) => {
180     // NICHT an lokalerFortschritt gekoppelt wie die Routenlinien: Badges
181     // sind Klickziele und müssen von Anfang an alle da sein.
182     let labelAlpha = (zoomedKapitel && kapitelNr !== zoomedKapitel)
183         ? alpha * (1 - kapitelZoomAmount)
184         : alpha;
185     // ACHTUNG Schwelle 1, nicht 0: kapitelZoomAmount läuft per lerp() nur
186     // asymptotisch, labelAlpha wird nie 0. Bei konstantem Alpha überspringt
187     // p5 das fill(), und fillText() erbt die Deckkraft, die kreisgrafik.js
188     // direkt in fillStyle geschrieben hat – Badges bleiben sichtbar.
189     if (labelAlpha < 1) return;
190
191     let start = lonLatToScreen(punkte[0][0], punkte[0][1], bbox, 0, 0); // zentrierte
Übersichtskarte, kein mapOffsetX
192     // Beim gezoomten Kapitel auf den echten Routenanfang überblenden – der

```

```

193 // Startpunkt der Übersichtslinie trifft ihn im Zoom nicht.
194 if (kapitelNr === zoomedKapitel && kapitelZoomAmount > 0.001) {
195     let daten = datenFuerKapitel(kapitelNr);
196     let routenLinie = (daten && daten.routenPfadDetail && daten.routenPfadDetail.length > 1)
197         ? daten.routenPfadDetail
198         : (daten && daten.routenPunkte);
199     if (routenLinie && routenLinie.length > 0) {
200         let praezise = lonLatToScreen(routenLinie[0][0], routenLinie[0][1], bbox, mapOffsetX,
mapOffsetY);
201         start = {
202             x: lerp(start.x, praezise.x, kapitelZoomAmount),
203             y: lerp(start.y, praezise.y, kapitelZoomAmount),
204         };
205     }
206 }
207
208 // Fester Platz auf einem kleinen Kreis um den echten Punkt, je Gruppe.
209 let dupGruppe = startDupGruppen[dupSchluessel[kapitelNr]];
210 let dupAnker = null;
211 let labellLinks = false; // Nummer nach links statt rechts setzen
212 if (dupGruppe.length > 1) {
213     let dupWinkel = (dupGruppe.indexOf(kapitelNr) / dupGruppe.length) * TWO_PI;
214     const dupVersatz = 13; // px
215     dupAnker = start; // echter Routenanfang, für die Verbindungslinie unten
216     start = {
217         x: start.x + cos(dupWinkel) * dupVersatz,
218         y: start.y + sin(dupWinkel) * dupVersatz,
219     };
220     // Nummer auf die Aussenseite, sonst steht sie bei den linken Badges
221     // quer über der Gruppenmitte.
222     labellLinks = cos(dupWinkel) < -0.01;
223 }
224
225 // Nur klickbar, solange kein anderes Kapitel offen ist.
226 let klickbar = kapitelHatEigeneAnsicht(kapitelNr) && !zoomedKapitel;
227 let hover = klickbar && dist(mouseX, mouseY, start.x, start.y) < FOTO_MARKER_TREFFER_RADIUS;
228 if (hover) kapitelHover = kapitelNr;
229
230 // Punkt und Nummer nehmen dieselbe Hitze wie die Linie.
231 let scheibe = kapitelScheiben()[i];
232 let lokalerFortschritt = scheibe
233     ? constrain(map(fortschritt, scheibe.von, scheibe.bis, 0, 1), 0, 1) : 0;
234 let hitze = (!zoomedKapitel && lokalerFortschritt > 0) ? kapitelHitze(fortschritt, scheibe) : 0;
235
236 // Verbindungsstrich vom versetzten Badge zum echten Routenanfang, sonst
237 // wirkt es wie ein Punkt ohne Route.
238 if (dupAnker) {
239     stroke(33, 43, 46, labelAlpha * 0.55);
240     strokeWeight(1);
241     line(dupAnker.x, dupAnker.y, start.x, start.y);
242     noStroke();
243 }
244
245 // ACHTUNG fill(hexString, alpha) ist keine verlässliche p5-Signatur –
246 // deshalb hier wie überall RGB statt Hex+Alpha.
247 if (hover) fill(FWERT_COLOR_RGB.r, FWERT_COLOR_RGB.g, FWERT_COLOR_RGB.b, labelAlpha); // #C2511C
248 else fill(lerp(33, FWERT_COLOR_RGB.r, hitze), lerp(43, FWERT_COLOR_RGB.g, hitze),
249     lerp(46, FWERT_COLOR_RGB.b, hitze), labelAlpha); // #212B2E .. #C2511C
250 ellipse(start.x, start.y, hover ? 11 : 8, hover ? 11 : 8);
251 // p5s text() bleibt beim Scrollen manchmal unsichtbar – direkt über den
252 // Canvas-Context, fillStyle kommt vom fill() oben.
253 if (labellLinks) {
254     textAlign(RIGHT, CENTER);
255     drawingContext.textAlign = 'right';
256     drawingContext.fillText(kapitelNr, start.x - 8, start.y);
257     textAlign(LEFT, CENTER);

```

```

258     drawingContext.textAlign = 'left';
259   } else {
260     drawingContext.fillText(kapitelNr, start.x + 8, start.y);
261   }
262   });
263
264   // Kapitel 1 hat keine Übersichtsroute – Nummer hier ergänzt, Klick scrollt
265   // zurück statt zu zoomen. Blendet im Kapitel-Zoom mit aus wie 02–18.
266   let ch1Alpha = zoomedKapitel ? alpha * (1 - kapitelZoomAmount) : alpha;
267   if (ch1Alpha >= 1) {
268     let ch1Start = lonLatToScreen(stationenData.routenPunkte[0][0], stationenData.routenPunkte[0]
[1], bbox, 0, 0); // zentrierte Übersichtskarte, kein mapOffsetX
269     let ch1Hover = dist(mouseX, mouseY, ch1Start.x, ch1Start.y) < FOTO_MARKER_TREFFER_RADIUS;
270     if (ch1Hover) kapitelHover = '01';
271     if (ch1Hover) fill(FWERT_COLOR_RGB.r, FWERT_COLOR_RGB.g, FWERT_COLOR_RGB.b, ch1Alpha); //
#C2511C
272     else fill(33, 43, 46, ch1Alpha); // #212B2E
273     ellipse(ch1Start.x, ch1Start.y, ch1Hover ? 11 : 8, ch1Hover ? 11 : 8);
274     drawingContext.fillText('01', ch1Start.x + 8, ch1Start.y); // siehe Kommentar oben (p5s text()-
Bug)
275   }
276
277   textStyle(NORMAL);
278   cursor(kapitelHover ? HAND : ARROW);
279
280
281   // Für die Annotationsbox in draw(); Kapitel 1 läuft dort über annIndex.
282   return { aktuelleAnnotationZoom };
283 }
284
285 // Zurück in die Kapitel-1-Ansicht; schliesst einen offenen Kapitel-Zoom mit.
286 function scrolleZuKapitel1() {
287   schliesseKapitelZoom();
288   let trackEl = document.querySelector('.scroll-track');
289   let ziel = trackEl.offsetHeight * SCROLL_MEILENSTEINE.zoomEnd;
290   window.scrollTo({ top: ziel, behavior: 'smooth' });
291 }
292
293 // Setzt Ansichtsmodus und Play-Animation zurück, damit jede Kapitel-Ansicht
294 // in der Kartenansicht startet. Kein Guard, anders als setzeKapitelAnsichtModus.
295 function setzeKapitelAnsichtZurueck() {
296   // Jedes Modul setzt seinen eigenen Zustand zurück. Ton stoppt in
297   // setzeGrafikZurueck() mit.
298   setzeAnsichtsModus('karte'); // sketch.js
299   setzeGrafikZurueck(); // spine-horizontal.js
300   // Startzeit für den Einstiegstext-Fade, bei jedem Kapitelwechsel neu.
301   starteKapitelEinstieg(); // sketch.js
302 }
303
304 // ACHTUNG genau einmal pro Frame ticken – deshalb ruft draw() direkt und
305 // nicht zeichneUebersichtsrouten(), die beim Ausblenden nicht mehr läuft.
306 function aktualisiereKapitelZoom() {
307   kapitelZoomAmount = lerp(kapitelZoomAmount, zoomedKapitel ? 1 : 0, 0.08);
308 }
309
310 // ACHTUNG setzt voraus, dass die Scrollposition schon im uebersichtRouten-Akt liegt –
311 // sonst schliesst der <=0-Check in draw() den Zoom im nächsten Frame
312 // wieder. Von aussen immer über springeZuKapitelZoom().
313 function oeffneKapitelZoom(nr) {
314   if (!kapitelHatEigeneAnsicht(nr)) return;
315   zoomedKapitel = nr;
316   setzeKapitelAnsichtZurueck();
317 }
318
319 function schliesseKapitelZoom() {
320   zoomedKapitel = null;

```

```

321     setzeKapitelAnsichtZurueck();
322 }
323
324 // Springt kurz hinter den Akt-Anfang und öffnet dort den Zoom, damit es von
325 // jeder Scrollposition aus geht. ACHTUNG ohne "smooth": sonst laufen Frames
326 // mit alter Position, und der <=0-Check setzt zoomedKapitel wieder auf null.
327 function springeZuKapitelZoom(nr) {
328     if (!kapitelHatEigeneAnsicht(nr)) return;
329     let trackEl = document.querySelector('.scroll-track');
330     let start = SCROLL_MEILENSTEINE.uebersichtRoutenStart
331         + 0.01 * (SCROLL_MEILENSTEINE.uebersichtRoutenEnd - SCROLL_MEILENSTEINE.uebersichtRoutenStart);
332     loeseKapitelKlemme(); // sonst zöge draw() sofort ans Kapitel-1-Ende zurück
333     window.scrollTo(0, trackEl.offsetHeight * start);
334     oeffneKapitelZoom(nr);
335 }
336
337 // "Alle"-Button. Zielt auf die MITTE des Akts (Routen gewachsen), nicht wie
338 // springeZuKapitelZoom auf den Anfang.
339 function springeZurUebersicht() {
340     let trackEl = document.querySelector('.scroll-track');
341     let mitte = (SCROLL_MEILENSTEINE.uebersichtRoutenStart + SCROLL_MEILENSTEINE.uebersichtRoutenEnd)
342 / 2;
343     loeseKapitelKlemme(); // sonst zöge draw() sofort ans Kapitel-1-Ende zurück
344     window.scrollTo(0, trackEl.offsetHeight * mitte);
345     schliesseKapitelZoom();
346 }
347
348 // --- Export -----
349 // Sieben Funktionen als Wert.
350 window.kapitelScheiben = kapitelScheiben;
351 window.zeichneUebersichtsrouten = zeichneUebersichtsrouten;
352 window.aktualisiereKapitelZoom = aktualisiereKapitelZoom;
353 window.scrolleZuKapitel1 = scrolleZuKapitel1;
354 window.schliesseKapitelZoom = schliesseKapitelZoom;
355 window.springeZuKapitelZoom = springeZuKapitelZoom;
356 window.springeZurUebersicht = springeZurUebersicht;
357
358 // Lesebindung statt Wertkopie: alle drei werden laufend umgeschaltet, eine
359 // Kopie nagelte die Leser auf null bzw. 0 fest. Schreiben von aussen wirkt nicht.
360 ['zoomedKapitel', 'kapitelZoomAmount', 'kapitelHover'].forEach(function (name) {
361     Object.defineProperty(window, name, {
362         get: function () {
363             return name === 'zoomedKapitel' ? zoomedKapitel
364                 : name === 'kapitelZoomAmount' ? kapitelZoomAmount
365                 : kapitelHover;
366         },
367         configurable: true,
368     });
369 });
370
371 })(); // Ende der Modulkapselung, siehe Kommentar oben

```